

A functional approach to differential programming in machine learning

MS22

SciCADE 2026

Michael Arbel
Research Scientist at INRIA

June 30, 2026

Functional Bilevel Optimization

Definition

Let $\mathcal{U}$ and $\mathcal{H}$ be Hilbert spaces and $L, G : \mathcal{U} \times \mathcal{H} \rightarrow \mathbb{R}$ two smooth functionals, with $P \mapsto G(U, P)$ strongly convex for all $U \in \mathcal{U}$. Consider the problem

$$\min_{U \in \mathcal{U}} J(U) = L(U, P_U)$$

s.t.

$$P_U = \arg \min_{P \in \mathcal{H}} G(U, P)$$

Functional Bilevel Optimization

Definition

Let $\mathcal{U}$ and $\mathcal{H}$ be Hilbert spaces and $L, G : \mathcal{U} \times \mathcal{H} \rightarrow \mathbb{R}$ two smooth functionals, with $P \mapsto G(U, P)$ strongly convex for all $U \in \mathcal{U}$. Consider the problem

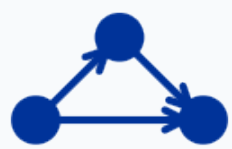
$$\min_{U \in \mathcal{U}} J(U) = L(U, P_U)$$

s.t.

$$P_U = \arg \min_{P \in \mathcal{H}} G(U, P)$$

Causal effect estimation

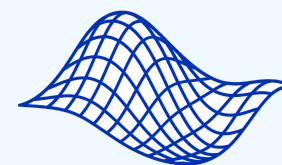
Goal: Estimate treatment effect



Causal model **constrains** prediction

Data-driven PDE discovery

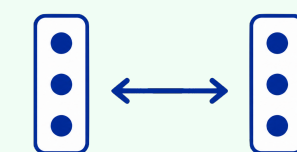
Goal: Learn unknown PDE



Variational PDE **constrains** the state

Self-supervised learning

Goal: Learn data encoder from views



Regression **constrains** prediction

Functional Bilevel Optimization

Definition

Let $\mathcal{U}$ and $\mathcal{H}$ be Hilbert spaces and $L, G : \mathcal{U} \times \mathcal{H} \rightarrow \mathbb{R}$ two smooth functionals, with $P \mapsto G(U, P)$ strongly convex for all $U \in \mathcal{U}$. Consider the problem

$$\min_{U \in \mathcal{U}} J(U) = L(U, P_U)$$

s.t.

$$P_U = \arg \min_{P \in \mathcal{H}} G(U, P)$$

- L and G might depend on data (learning problem).
- $\mathcal{U}$ and $\mathcal{H}$ might be infinite dimensional spaces of functions
- Elements $U \in \mathcal{U}$ and $P \in \mathcal{H}$ might be functions of high-dimensional vectors

How to solve FBO in practice?

Need function approximation
e.g., neural networks

I- Functional Implicit Differentiation

**II- Self-supervised learning as
functional bilevel optimization**

I- Functional Implicit Differentiation

Direct neural approximation fails to solve ICL problems reliably

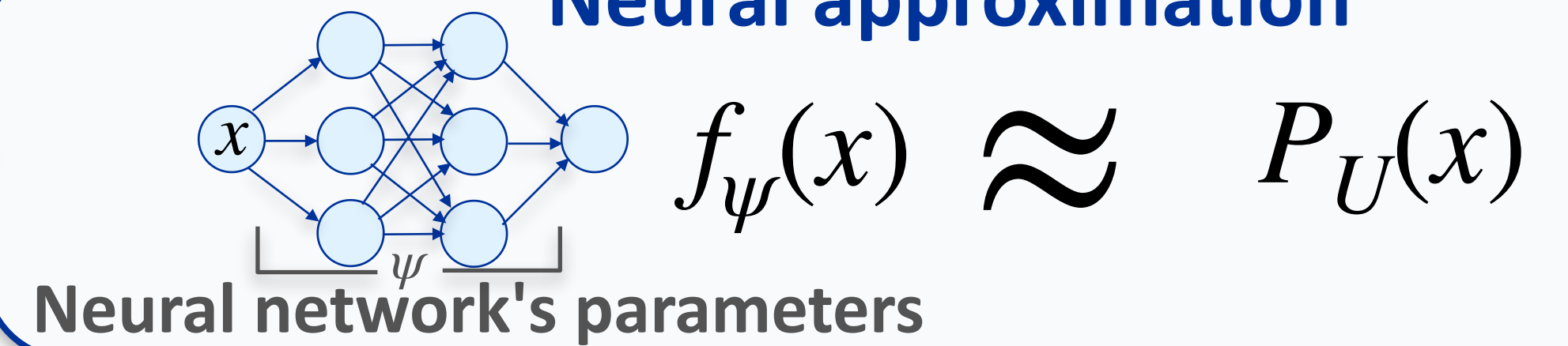
FBO problem

$$\min_{U \in \mathcal{U}} J(U) = L(U, P_U) \quad \text{s.t.} \quad P_U = \arg \min_{P \in \mathcal{H}} G(U, P)$$

function of high-dimensional input x

How?

Neural approximation



Direct neural approximation fails to solve ICL problems reliably

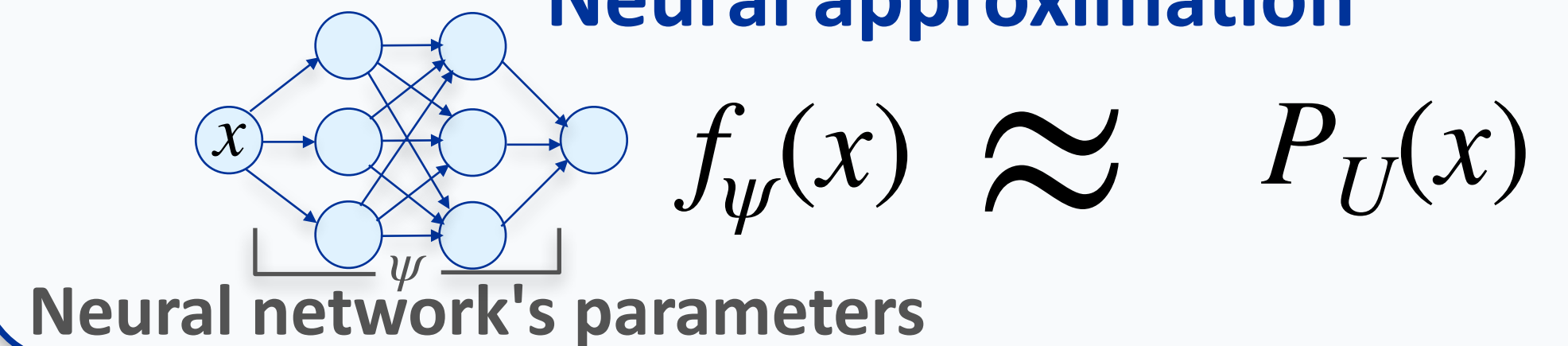
FBO problem

$$\min_{U \in \mathcal{U}} J(U) = L(U, P_U) \quad \text{s.t.} \quad P_U = \arg \min_{P \in \mathcal{H}} G(U, P)$$

function of high-dimensional input x

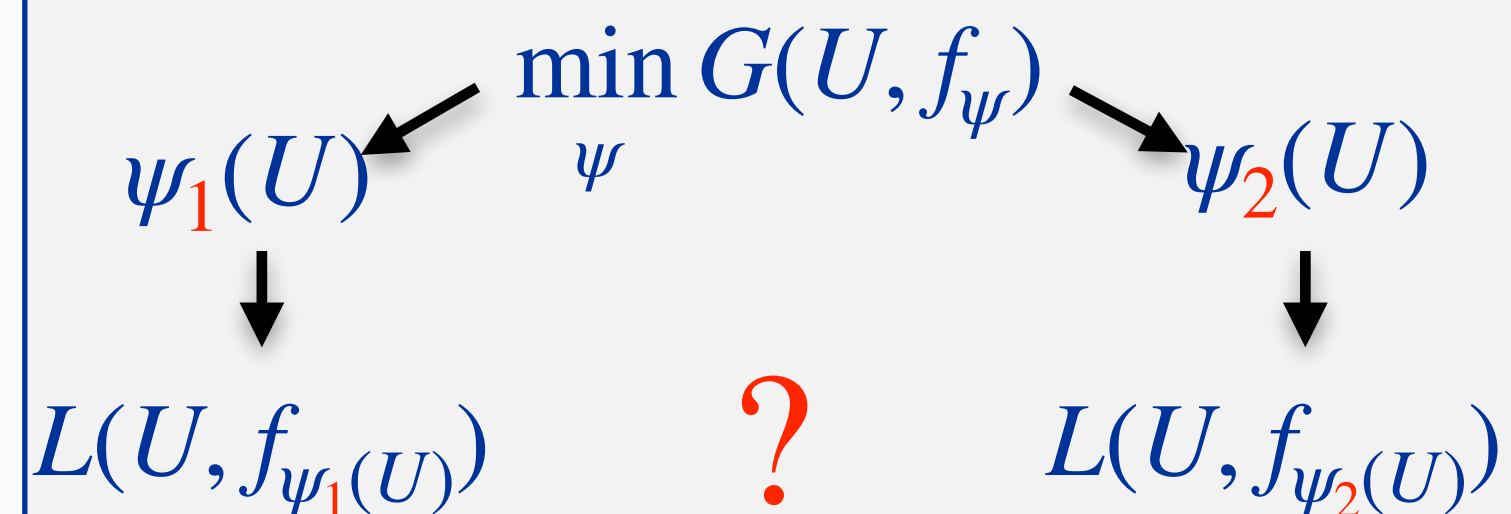
How?

Neural approximation



Differentiable programming

Solve ill-defined surrogate problem



Network overparameterization

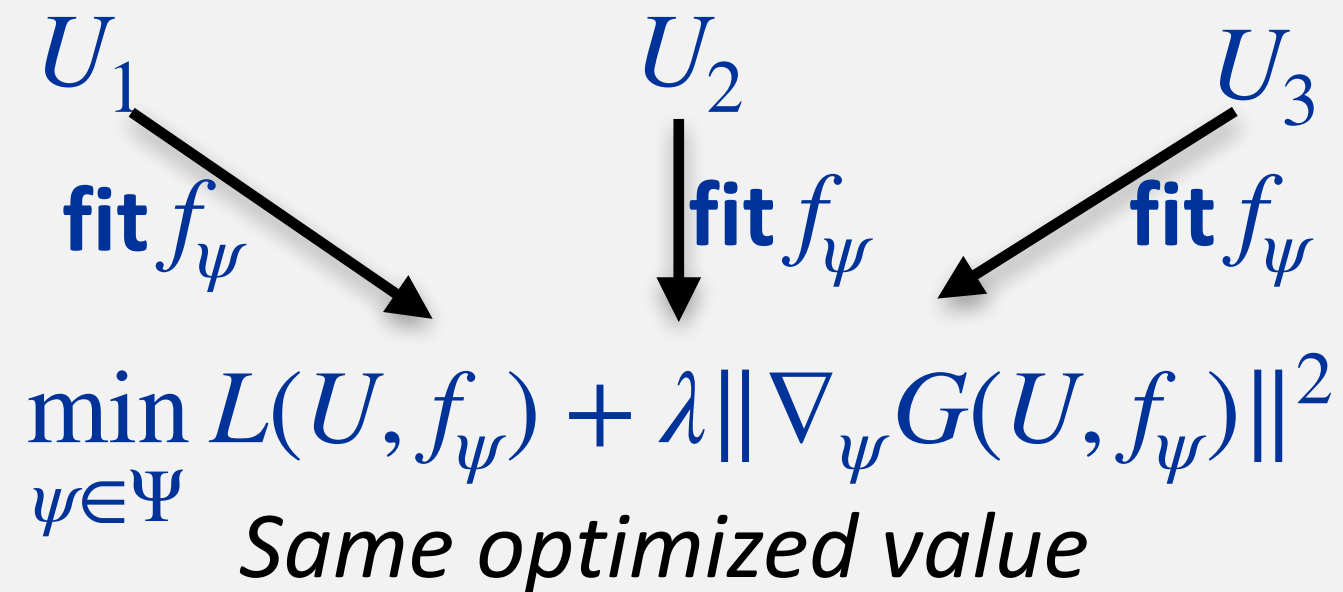


Unreliable estimate of U

[Vicol et al. 2021, ICML]

Penalty methods

Merge objective and constraint



Expressive f_{ψ} can fit both L and G

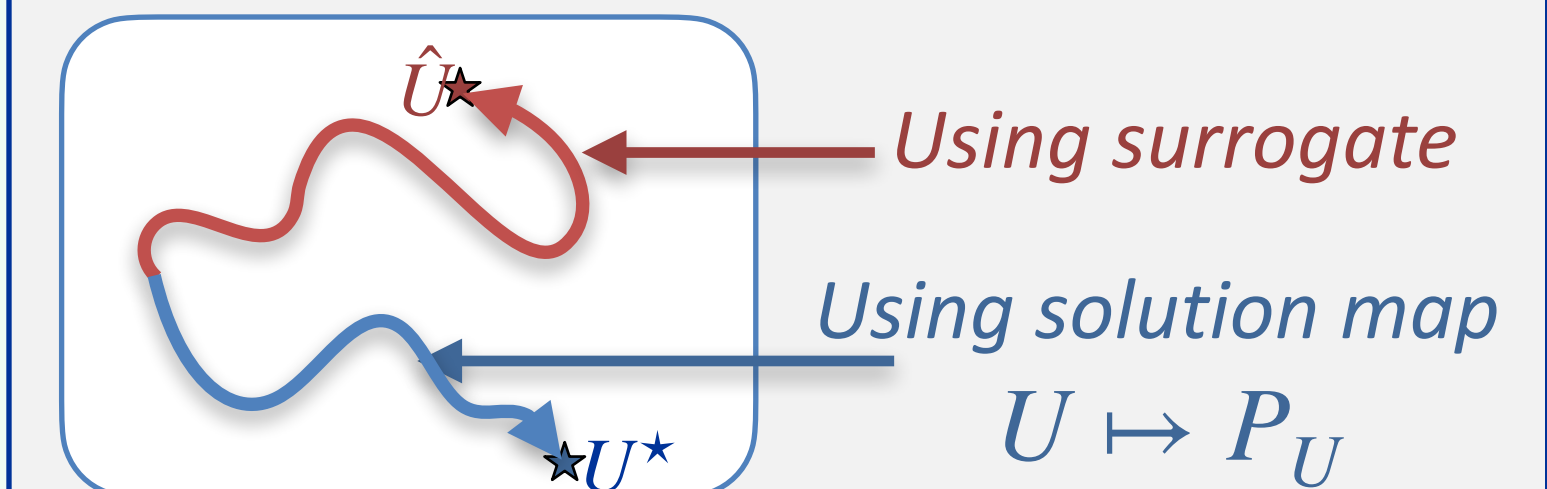


U can no longer be estimated

[Krishnapriyan et al. 2021, NeurIPS]

Neural operators

Learn surrogate of full solution map



Surrogate trained by average-fit over U



Bad U -updates despite good fit

[Elmachtoub & Grigas, 2022, MS]

The curse of network overparameterization

The curse of irrelevant accuracy

FuncBO (NeurIPS 2024)

Functional Bilevel Optimization for Machine Learning

Ieva Petrulionyte, Julien Mairal, Michael Arbel

Univ. Grenoble Alpes, Inria, CNRS, Grenoble INP, LJK, 38000 Grenoble, France

`firstname.lastname@inria.fr`



Follow up: Petrulionyte et al, [Non-Stationary Functional Bilevel Optimization](#), AISTATS 2026

FuncBO: Differentiate first, then approximate

FBO problem

$$\min_{U \in \mathcal{U}} J(U) = L(U, P_U)$$

s.t.

$$P_U = \arg \min_{P \in \mathcal{H}} G(U, P)$$

Standard differentiable programming approach

Neural approximation *of inner solution P*

yields surrogate problem

Differentiation
in parameter space *of surrogate objective*



Ill-defined surrogate problem

FunBO's approach: Functional Implicit Differentiation

Differentiation
in functional space *of FBO objective*

yields functional gradient

Neural approximation *of gradient components
(e.g., predictor sensitivity)*



Well-defined ICL gradient

Implicit differentiation in L_2 spaces and adjoint sensitivity method

FBO problem

$$\min_{U \in \mathcal{U}} J(U) = L(U, P_U)$$

s.t.

$$P_U = \arg \min_{P \in \mathcal{H}} G(U, P)$$

$$L(U, P) = \mathbb{E}[\ell(U, P(x), y)]$$

$$G(U, P) = \mathbb{E}[g(U, P(x), y)]$$

- Inner and outer objectives are expectations of point-wise losses under a data distribution $\mathbb{P}$
- $\mathcal{U}$ is a Euclidean space, $\mathcal{H}$ is space of square integrable functions under data distribution $\mathbb{P}$

Implicit differentiation in L_2 spaces and adjoint sensitivity method

FBO problem

$$\min_{U \in \mathcal{U}} J(U) = L(U, P_U)$$

s.t.

$$P_U = \arg \min_{P \in \mathcal{H}} G(U, P)$$

$$L(U, P) = \mathbb{E}[\ell(U, P(x), y)]$$

$$G(U, P) = \mathbb{E}[g(U, P(x), y)]$$

Proposition (informal)

If $p \mapsto g(U, p, y)$ is strongly convex + regularity, integrability and growth assumptions:

$$\nabla J(U) = F(U, P_U, a_U) \triangleq \mathbb{E} \left[\partial_1 \ell(U, P_U(x), y) + \partial_{1,2} g(U, P_U(x), y) a_U(x) \right]$$

where the a_U is the unique minimizer in $L_2(\mathbb{P})$ of the objective: $a \mapsto L_{adj}(U, P_U, a)$

$$L_{adj}(U, P, a) = \mathbb{E} \left[\frac{1}{2} a(x)^\top \partial_{2,2} g(U, P(x), y) a(x) + \partial_2 \ell(U, P(x), y) a(x) \right]$$

The FuncBO algorithm

FBO problem

$$\min_{U \in \mathcal{U}} J(U) = L(U, P_U)$$

s.t.

$$P_U = \arg \min_{P \in \mathcal{H}} G(U, P)$$

Differentiation in functional space

$$\nabla J(U) = F(U, P_U, a_U)$$

$$a_U = \arg \min_{a \in \mathcal{H}} L_{adj}(U, P_U, a)$$

Neural approximation

$$f_\psi \approx \arg \min_{f_\psi \in NN} \hat{G}(U, f_\psi) \approx P_U$$

$$g_\theta \approx \arg \min_{g_\theta \in NN} \hat{L}_{adj}(U, f_\psi, g_\theta) \approx a_U$$

$$\hat{F}(U, f_\psi, g_\theta) \approx \nabla J(U)$$

- $\hat{F}$, $\hat{G}$ and $\hat{L}_{adj}$ are empirical averages to estimate F , G and L_{adj}
- f_ψ and g_θ obtained by gradient descent w.r.t the parameters ψ and θ
- U updated iteratively using gradient estimate $\hat{F}(U, f_\psi, g_\theta)$

The FuncBO algorithm

FBO problem

$$\min_{U \in \mathcal{U}} J(U) = L(U, P_U)$$

s.t.

$$P_U = \arg \min_{P \in \mathcal{H}} G(U, P)$$

Differentiation in functional space

$$\nabla J(U) = F(U, P_U, a_U)$$

$$a_U = \arg \min_{a \in \mathcal{H}} L_{adj}(U, P_U, a)$$

Neural approximation

$$f_\psi \approx \arg \min_{f_\psi \in NN} \hat{G}(U, f_\psi) \approx P_U$$

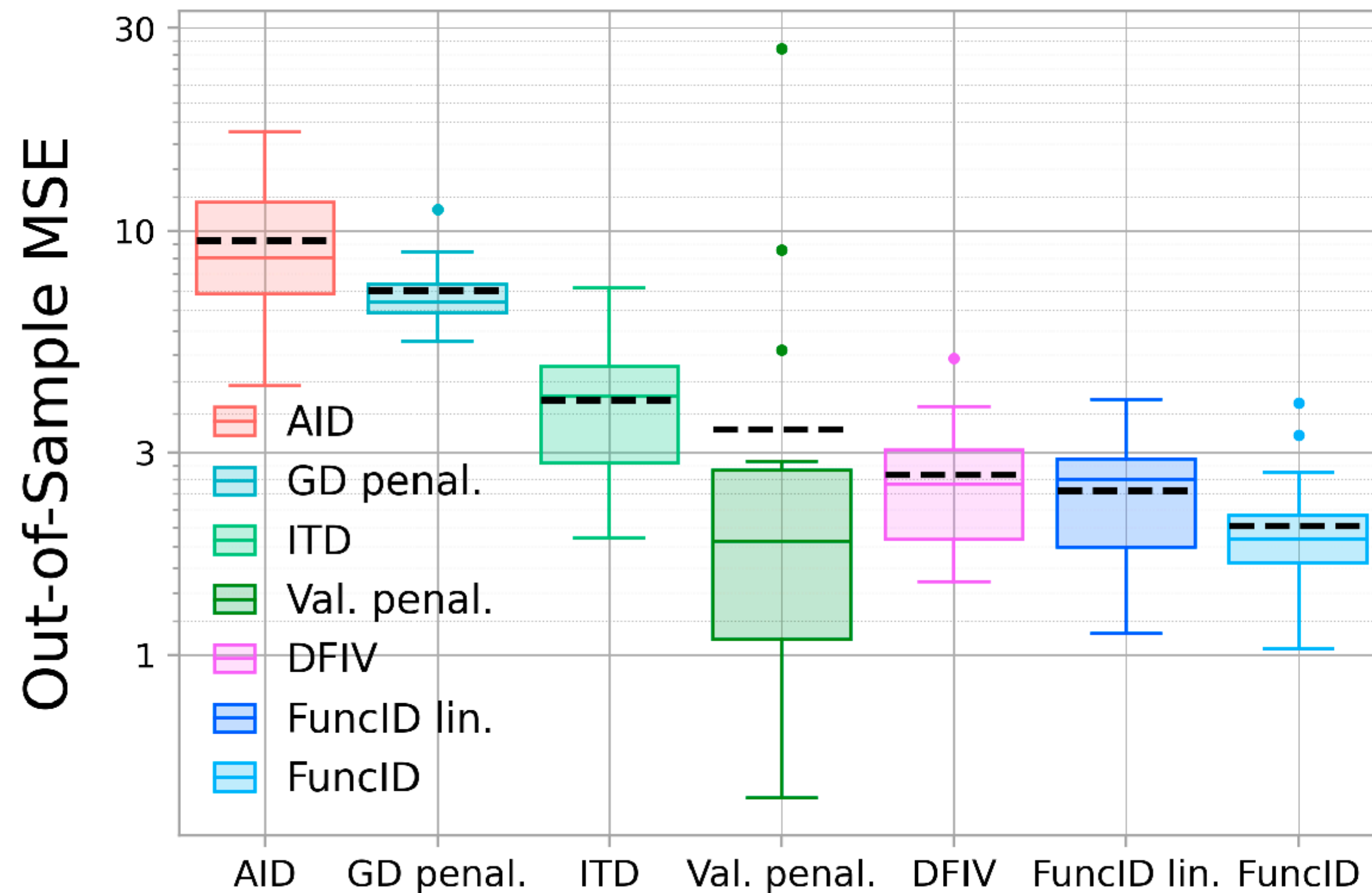
$$g_\theta \approx \arg \min_{g_\theta \in NN} \hat{L}_{adj}(U, f_\psi, g_\theta) \approx a_U$$

$$\hat{F}(U, f_\psi, g_\theta) \approx \nabla J(U)$$

- **Memory and time savings:** No need for higher-order derivatives w.r.t network parameters
- **Stability:** Strongly-convex adjoint objective in function space $\longrightarrow$ Well-defined gradient

Application to causal effect estimation

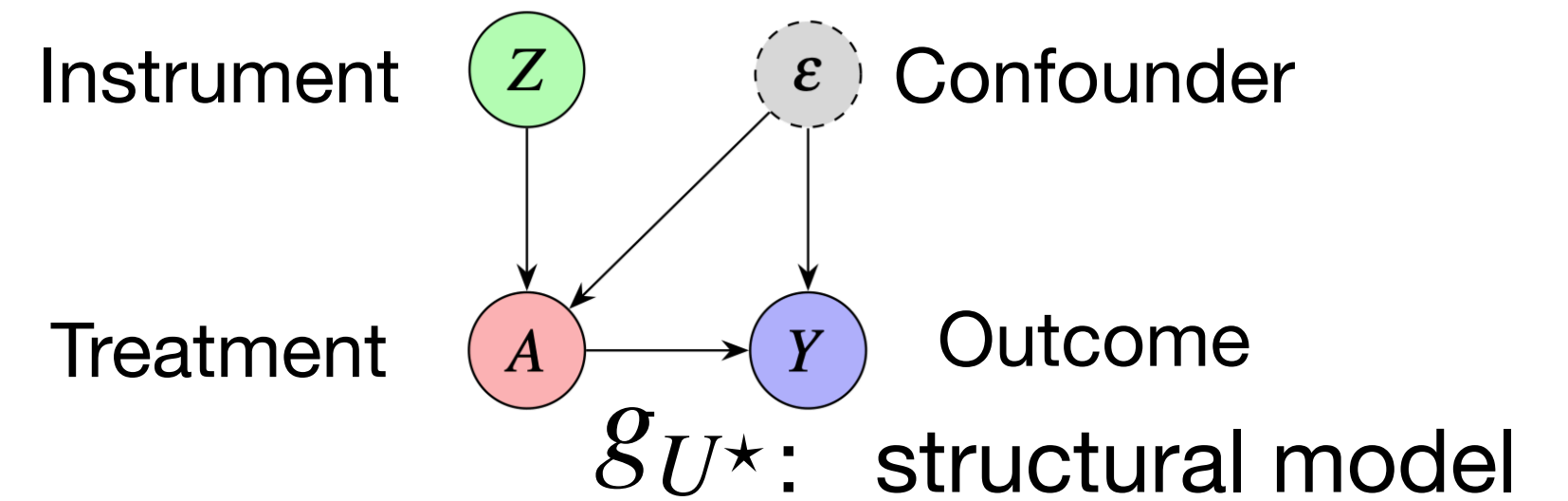
Results on synthetic task from [Xu et al. 2020]



- Large improvement over classical BO algorithms (AID/ITD)
- Competitive with problem specific method (DFIV)

Causal effect estimation

Fit U using variation induced by Z



$$\min_{U \in \mathcal{U}} \mathbb{E} [\|Y - P_U(Z)\|^2] \leftarrow L(U, P_U)$$

Instrument-induced
Projection P_U

Project $g_U(A)$ onto functions of Z

$$P_U = \arg \min_{P \in L_2(\mathbb{P})} \mathbb{E} [\|P(Z) - g_U(A)\|^2]$$

$G(U, P)$

What next?

- Convergence/generalization results for FuncBO? →

NeurIPS 2025

Learning Theory for Kernel Bilevel Optimization

Fares El Khoury¹ Edouard Pauwels² Samuel Vaite³ Michael Arbel¹



Next step: dealing with neural networks
(coming soon)

- **Future work:** Extend to when the inner problem involves a PDE

Data driven PDE discovery

$$\min_{\theta \in \Theta} J(\theta) = \mathbb{E} \left[|P_{\theta}(t, x) - \mathcal{O}(t, x)|^2 \right]$$

Measurements
e.g., temperature, pressure

s.t. P_{θ} solves

$$\partial_t P = \mathcal{D}(P) + \mathcal{C}_{\theta}(P)$$

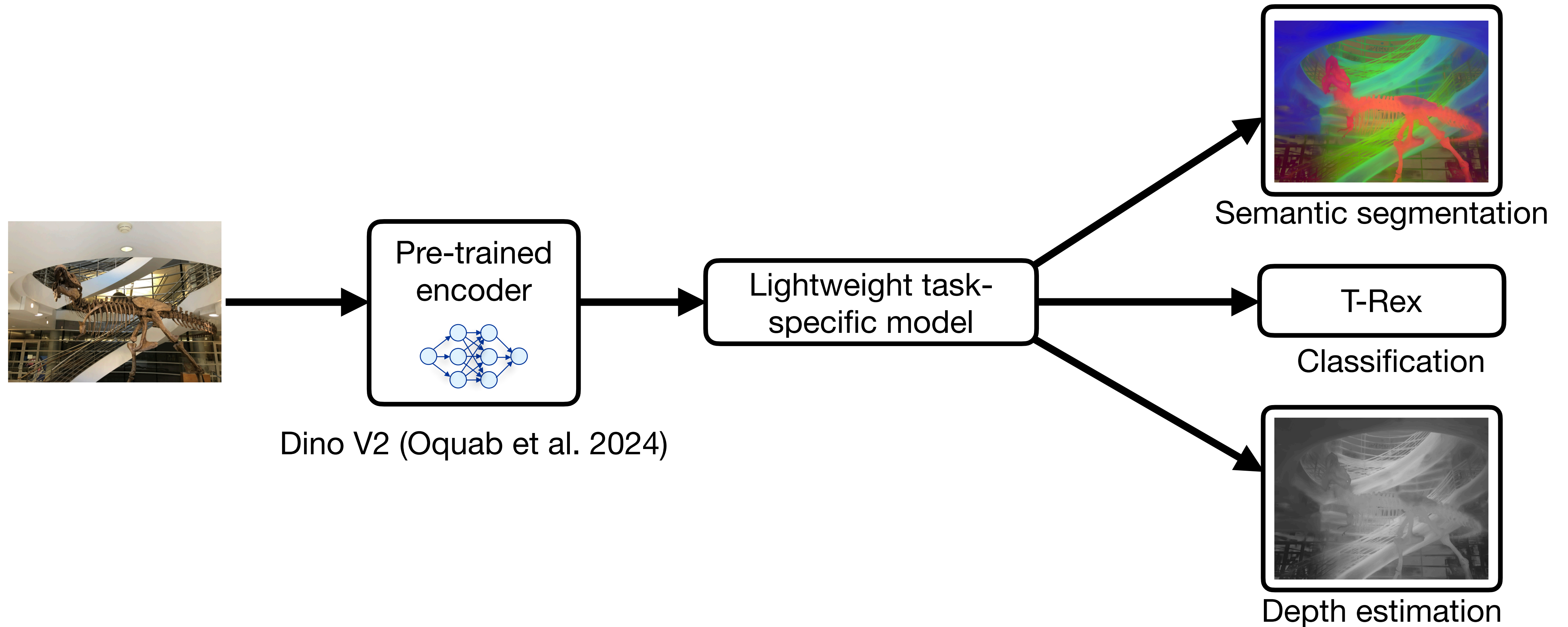
Differentiable climate model

e.g. Learnable closure model

II- Self-supervised learning as functional bilevel optimization

Context and motivation

Self-supervised pre-training can produce highly transferable visual features



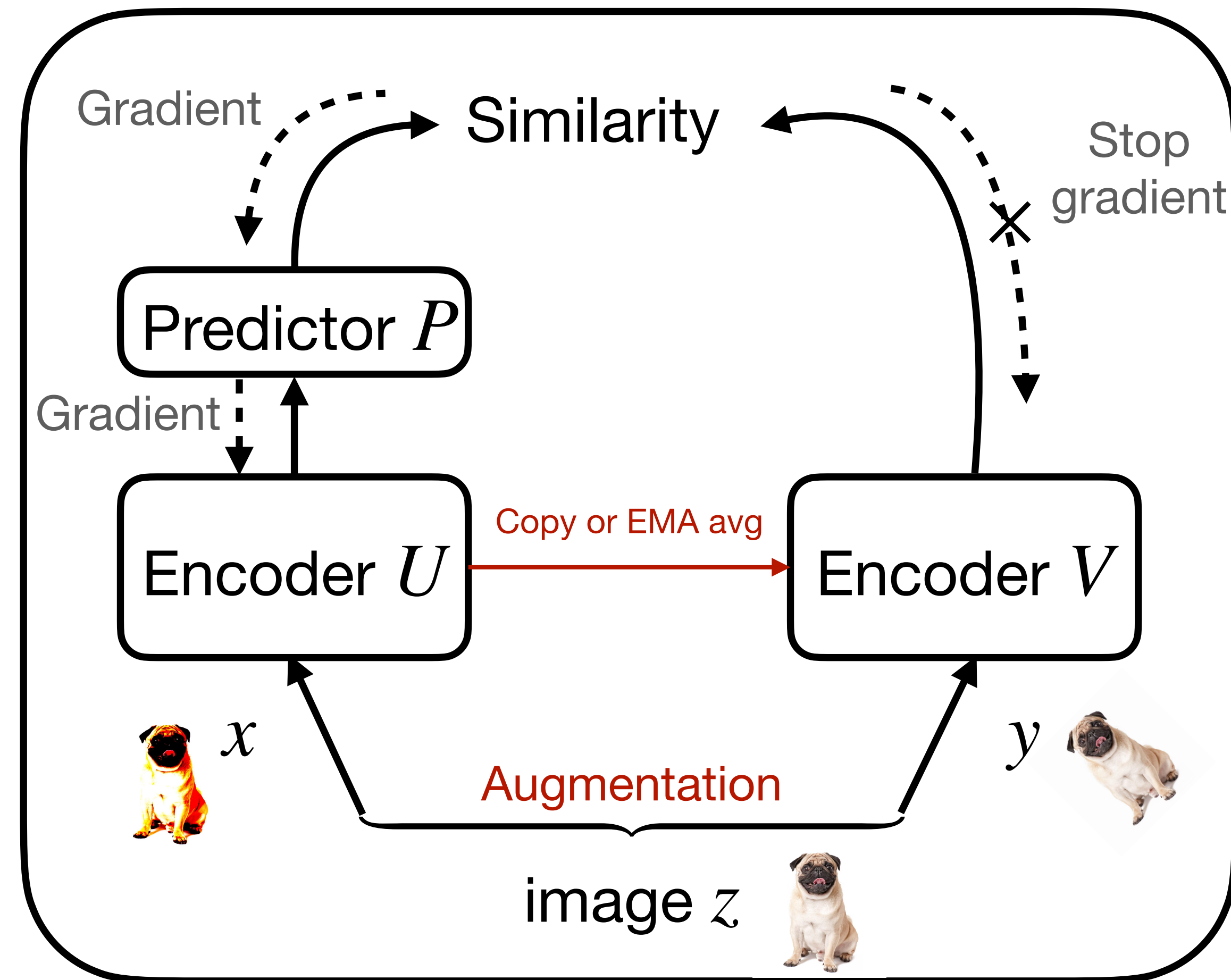
Yet, why these features emerge and what structure they capture is poorly understood

Self-distillation methods for SSL: A puzzle

- Not an optimization algorithm [Ponce et al, 2026]
- Can collapse to trivial representation
- Uses heuristics to avoid collapse (Stop-gradient, EMA)
- Excellent empirical performance (e.g., Dino v1, v2, v3)

Goal: Understanding:

- What is learned?
- When does collapse occur?
- How to reliably prevent collapse? Bilevel optimization to the rescue!



Self-distillation SSL: Existing theory is informative but restricted

Special setting

- Linear encoders U, V
- Particular data covariance structure
- Special initialization (ex: orthogonal)

Tian et al. 2021

Littwin et al. 2024

Liu et al. 2022

Tang et al. 2023

?

General setting

- Non-linear encoders U, V
- General data distributions
- General initialization

PEIRA: Learning Predictive Encoders through Inter-View Regressor Alignment

Michael Arbel*

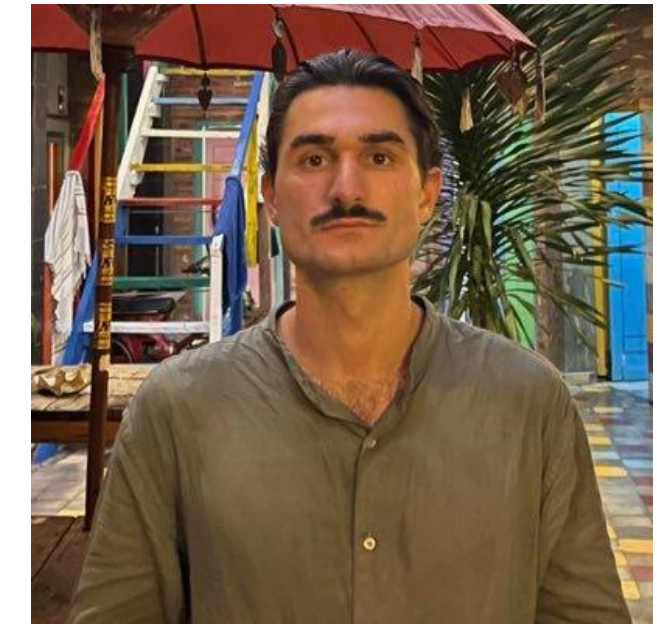
Univ. Grenoble Alpes, Inria
CNRS, Grenoble INP, LJK
michael.arbel@inria.fr

Basile Terver*

Ecole Normale Supérieure / PSL
Inria Paris
basile.terver@ens.psl.edu

Jean Ponce

Ecole Normale Supérieure / PSL
New York University
jean.ponce@ens.fr



- Characterization of a self-distillation dynamics for SSL
 - Limiting objects, convergence of the dynamics and characterization of collapse
- Bilevel optimization alternative to SSL that provably avoids collapse

A self-distillation dynamics in function space

$$\mathcal{L}(P, U, V) = \frac{1}{2} \mathbb{E}[\|PU(x) - V(y)\|^2] + \frac{\lambda}{2} \|P\|^2 + \frac{\lambda}{2} \mathbb{E}[\|U(x)\|^2]$$

$$\begin{cases} P^\star = \arg \min_P \mathcal{L}(P, U, V) \\ \dot{U} = -\partial_U \mathcal{L}(P^\star, U, V) \\ V = U \end{cases}$$

- Optimal linear predictor (as in Tang et al. 2023)
- No parametric form for the encoder U
- Dynamics defined in function space $L_2(\mathbb{P}_x, \mathbb{R}^k)$

What are the equilibria? Which ones are stable?

A self-distillation dynamics in function space

$$\mathcal{L}(P, U, V) = \frac{1}{2} \mathbb{E}[\|PU(x) - V(y)\|^2] + \frac{\lambda}{2} \|P\|^2 + \frac{\lambda}{2} \mathbb{E}[\|U(x)\|^2]$$

$$\begin{cases} P^\star = \arg \min_P \mathcal{L}(P, U, V) \\ \dot{U} = -\partial_U \mathcal{L}(P^\star, U, V) \\ V = U \end{cases}$$

- Optimal linear predictor (as in Tang et al. 2023)
- No parametric form for the encoder U
- Dynamics defined in function space $L_2(\mathbb{P}_x, \mathbb{R}^k)$

What are the equilibria? Which ones are stable?

Canonical Correlation Analysis to the rescue

Non-Linear Canonical Correlation Analysis (Michaeli et al. 2015)

Correlation maximization

$$\max_{(U,V) \in \mathcal{U} \times \mathcal{V}} \mathbb{E}[U(x)^\top V(y)]$$

$L_2(\mathbb{P}_x)^k$

$L_2(\mathbb{P}_y)^k$

s.t.

$$\mathbb{E}[U(x)U(x)^\top] = I$$

$$\mathbb{E}[V(y)V(y)^\top] = I$$

whitening constraints

Non-linear CCA solution: top-k canonical functions

$$U(x) = \begin{pmatrix} u_1(x) \\ \vdots \\ u_k(x) \end{pmatrix}, \quad V(y) = \begin{pmatrix} v_1(y) \\ \vdots \\ v_k(y) \end{pmatrix}$$

Canonical correlation $1 = s_1 \geq s_2 \geq \dots$

$$\mathbb{E}[u_i(x)v_i(y)] = s_i \qquad \mathbb{E}[u_i(x)v_j(y)] = 0$$

De-correlation (when $i \neq j$)

Orthonormal basis of $L_2(\mathbb{P}_y)$

Orthonormal basis of $L_2(\mathbb{P}_x)$

Self-distillation in function space: Convergence and Equilibria

Theorem (Informal): Assume x and y play symmetric roles and data regular enough.

Then: 1- For any initialization U_0 , the dynamics converges to an equilibrium $U^\star$.

2- Equilibria span CCA subspaces. The only stable ones are of the form:

$$U^\star(x) = \sum_{i=1}^r f(s_i, \lambda) q_i u_i(x)$$

Diagram illustrating the components of the equilibrium function $U^\star(x)$:

- Any $0 \leq r \leq k$** : Points to the index r in the summation.
- Regularization**: Points to the parameter λ in the function $f(s_i, \lambda)$.
- Any orthonormal family $(q_i)_{1 \leq i \leq r}$ of $\mathbb{R}^k$** : Points to the vector q_i .
- Canonical function**: Points to the function $u_i(x)$.
- Canonical correlation $s_1 \geq s_2 \geq \dots$** : Points to the scalar s_i .
- Spectral filter**: Points to the function $f(s, \lambda)$.

The spectral filter function is defined as:

$$f(s, \lambda) = \frac{1}{\sqrt{2}}(s + \sqrt{s^2 - 4\lambda}) \mathbf{1}_{s^2 \geq 4\lambda}$$

Collapse occurs when an admissible CCA direction ($f(s_i, \lambda) > 0$ for $i \leq k$) is not selected ($r < i$)

**Preventing collapse through
bilevel optimization**

Intuition: Optimal linear predictor as signal-to-noise ratio

- Optimal predictor between two views:

The diagram shows the formula for the optimal linear predictor $P^\star$. The formula is enclosed in a light green rounded rectangle. It consists of two main parts: a purple rounded rectangle on the left containing $\mathbb{E} [V(y)U(x)^\top]$, and a blue rounded rectangle on the right containing $\mathbb{E} [U(x)U(x)^\top]$. A red arrow points from the text "Shared signal" to the purple box. A blue arrow points from the text "Within view variability (noise)" to the blue box. A green arrow points from the text "Signal to noise ratio (matrix form)" to the entire expression in parentheses. The formula is
$$P^\star = \mathbb{E} [V(y)U(x)^\top] \left(\mathbb{E} [U(x)U(x)^\top] + \lambda I \right)^{-1}$$

$Tr(P^\star)$ acts as a scalar summary of the matrix signal to noise ratio

Predictive Embeddings through Inter-View Regressor Alignment

- Find optimal symmetrized regressor for each encoders pairs (U, V)

$$\frac{1}{2}\mathbb{E}[\|PU(x) - V(y)\|^2] + \frac{1}{2}\mathbb{E}[\|PV(y) - U(x)\|^2] + \frac{\lambda}{2}\|P\|^2$$

$$P_{U,V}^\star = \arg \min_{P \in \mathbb{R}^{k \times k}} \mathcal{L}_{reg}(P; U, V)$$

- Learn encoders by minimizing regularized negative trace of optimal regressor:

Negative trace minimization to align representation

Regularization to fix the scale

$$\mathcal{L}_{PEIRA}(U, V) = -\frac{1}{2}\text{Tr}(P_{U,V}^\star) + \frac{\lambda}{2}\mathbb{E}[\|V(x)\|^2 + \|U(y)\|^2]$$

PEIRA objective: Global maximizers

Theorem (Informal): Assume the data distribution admits a non-linear CCA.

Then: 1- Only global minimizers of $\mathcal{L}_{PEIRA}$ are stable.

2- They are exactly of the form:

$$(U^\star(x), V^\star(y)) = \sum_{i=1}^k f(s_i, \lambda) q_i (u_i(x), v_i(y))$$

Encoder's dimension $\rightarrow k$

Any orthonormal family $(q_i)_{1 \leq i \leq r}$ of $\mathbb{R}^k$

Soft-thresholding filter $\frac{1}{\sqrt{2}} \left(\max(\sqrt{s} - \lambda), 0 \right)^{\frac{1}{2}}$

Canonical functions $(u_i(x), v_i(y))$

Pretraining on ImageNet1k and Cifar10

Method	Approach	ImageNet-1K $\uparrow$	CIFAR-10 $\uparrow$
BYOL [Grill et al., 2020]	self-distillation (EMA)	69.3	–
SimSiam [Chen and He, 2021]	self-distillation (stop-gradient)	67.9	–
SimCLR [Chen et al., 2020]	contrastive (negatives)	68.5	–
VICReg [Bardes et al., 2022]	optimization	68.81 ± 0.09	90.92 ± 0.14
SIGReg [Balestriero and LeCun, 2025]	optimization	66.26 ± 0.28	92.34 ± 0.21
PEIRA (ours)	optimization	66.50 ± 0.02	90.97 ± 0.10

Competitive with some SOTA methods

Conclusion: Part II

Summary

- There is a formal connexion between self-distillation SSL methods and non-linear CCA
- Collapse is still an issue for these methods
- PEIRA leverages this connection with CCA while provably avoiding collapse
- Promising results on image representation

Future directions/open questions

- More applications? Video, time series, world-models
- Smarter algorithms? Avoiding full matrix inversion? Variance reduction, etc
- Statistical guarantees?

Thank you!